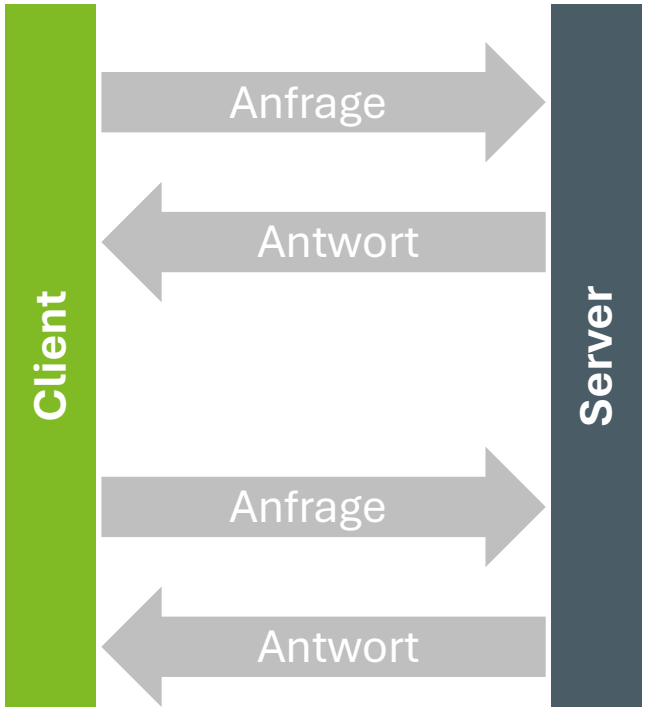
Request - Response

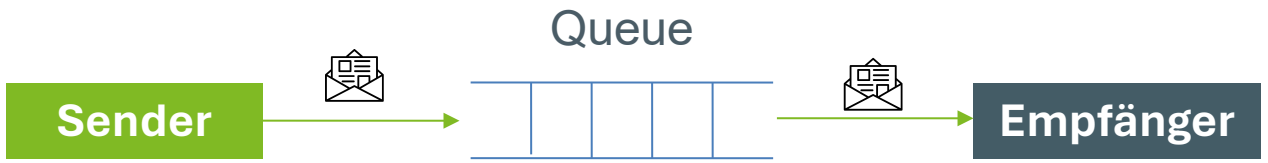
Kommunikationsmuster zum Austausch von Informationen zwischen einem Client und einem Server, bei dem der Client eine Anfrage (Request) sendet und der Server darauf mit einer Antwort (Response) reagiert.



- Logisch gesehen synchron, aber technisch beides möglich:
  - Synchron (z.B. Browser -> Web Server)
  - Asynchron (await fetch)

Messaging / Queuing

- Idee: Systeme kommunizieren durch Queues (Warteschlangen) asynchron miteinander

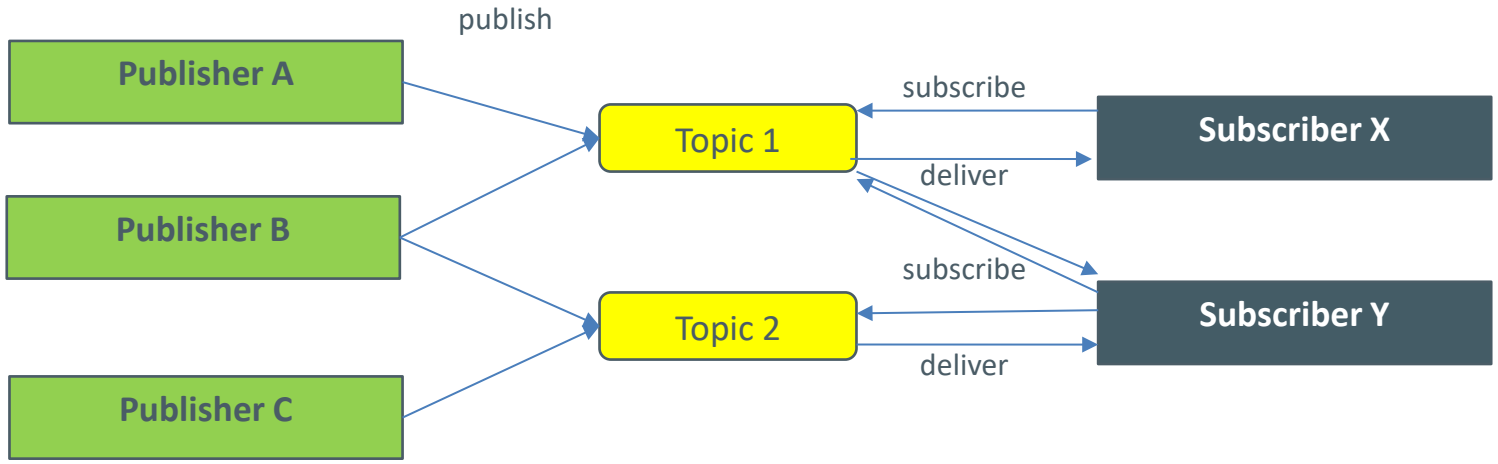


- Queues sind Dienste, die Nachrichten entgegennehmen, speichern und weitergeben.
- Queues entkoppeln Sender und Empfänger voneinander (zeitlich und logisch).
- Queues haben i.A. logische Namen zur Adressierung.

- Wie kann man damit Request-Response simulieren?

Publish-Subscribe

Analogie: Abonnieren eines YouTube-Kanals und Benachrichtigung, wenn ein neues Video zur Verfügung steht.



- Lose Kopplung von Sender und Empfänger
- Sehr flexibel
- Komplex und der Ablauf ist oft schwer nachvollziehbar

Fehler

Fehler-Kategorie

- Hardware-Fehler
  - Festplatten
  - Speicher
  - Netzwerk
  - …
- Software-Fehler
  Oft viel gravierender!



Fehlertyp

- **Crash Failure**
  Komponente arbeitet nicht mehr.
- **Omission Failure**
  Server antwortet nicht mehr auf Requests, bzw. schickt keine Nachrichten mehr.
- **Timing Failure**
  Antwort später als erwartet.
- **Response Failure**
  Antwort ist falsch.
- Arbitrary Failure (**Byzantine Failure**)
  Server produziert willkürlich Fehler.

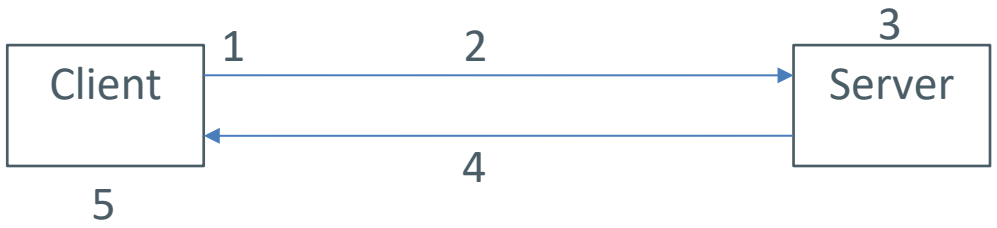
Redundanz

- Um Fehler zu verbergen, wird oft Redundanz genutzt
  - Redundante Daten (z.B. Prüfbits, Checksum, …)
  - zeitliche Redundanz: Automatic Retries (z.B. erneutes Versenden von Nachrichten)
  - Physische Redundanz: Dopplung wichtiger Komponenten (z.B. RAID-Platten)



Fehlerszenarien

Betrachten wir das Beispiel eines Aufrufs eines Web-Services. Was könnte alles schiefgehen?



Mögliche Fehler:

1. Client findet den Server nicht
2. Nachricht geht auf dem Weg zum Server verloren
3. Server stürzt nach Erhalt der Nachricht ab
4. Antwort geht auf dem Weg zum Client verloren
5. Client stürzt vor Erhalt der Antwort ab

Logische Uhren

- Da die Lichtgeschwindigkeit endlich ist, treten immer Latenzen auf.
- Diese kann man zwar schätzen, aber nicht exakt vorhersagen.
- Eine exakte Zeitsynchronisation gibt es nicht (nur im Film).

- Lösung: Jeder Rechner in einem verteilten System hat eine „logische Uhr“ (einen int).
- Sendet ein Rechner eine Nachricht an einen anderen, sendet er seinen Zeitstempel mit und der Empfänger nimmt das Maximum dieses Zeitstempels und des eigenen und addiert 1.
- Dadurch ist sichergestellt, dass die „Zeit“ des Absendens kleiner ist als die des Empfangens und die Zeit auf einem Rechner nicht rückwärts läuft.
- Das Verfahren ist ein berühmtes Resultat von Leslie Lamport.

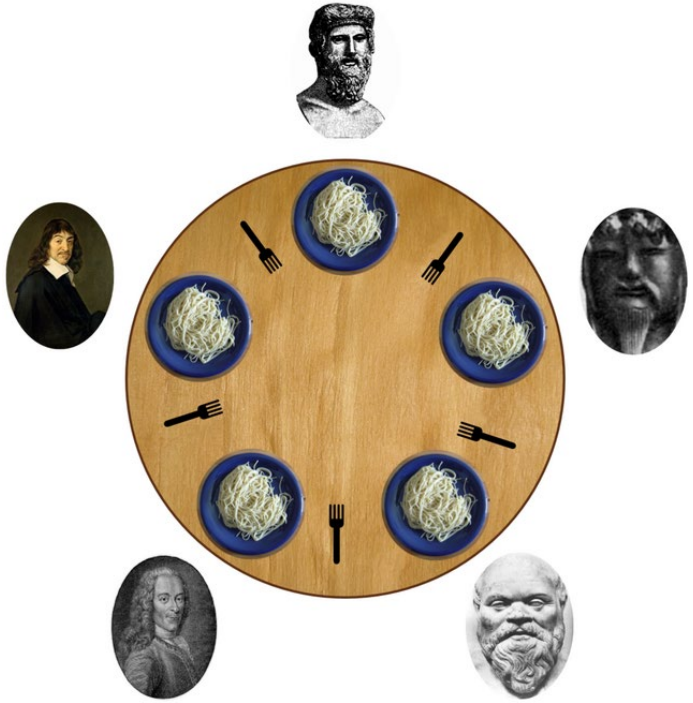


Deadlocks

Ein Deadlock ist eine Situation, in der Prozess wechselseitig auf sich warten und sich so gegenseitig blockieren.

Ein berühmtes Beispiel sind die „Dining Philosophers“ von Edsgar Dijkstra:

- Philosophen denken oder essen.
- Wenn sie essen wollen, müssen sie zuerst die linke und dann die rechte der vor ihnen liegenden Gabeln aufnehmen.



- Behauptung: Es können maximal zwei Philosophen gleichzeitig essen.
- Behauptung: Wenn alle Philosophen gleichzeitig essen wollen, verhungern sie alle.

Konsens

Ein weiteres häufiges Problem ist es, dass sich alle Prozesse auf einen gemeinsamen Wert einigen müssen („einen Konsens finden“).



Wie könnte ein Algorithmus aussehen, mit dem Teilnehmer sich auf einen Wert (Pizza ja/nein) einigen könnten?

Wer einen Vorschlag macht, sendet ihn an alle. Wenn die Mehrheit zustimmt, wird der Wert genommen.

Konsens

10 Sekunden später …



Der Algorithmus darf nicht zulassen, dass es verschiedene Mehrheiten gibt.
Wie verhindert man das?

Konsens

Am nächsten Tag …



Antworten kommen nicht, so dass es gar keinen Konsens gibt ?

Theorem: Wenn in einem asynchronen verteilten System Fehler auftreten können, gibt es keinen immer funktionierenden Konsens-Algorithmus!

Ausblick auf die Master-Vorlesung „Verteilte Systeme“

- Architekturen
- Kommunikation
  RPC, Messaging, Web Services, REST
- Koordination
  Zeit und Uhren, wechselseitiger Ausschluss
- Konsistenz und Replikation
- Fehlertoleranz
- Verteilte Dateisysteme
- Praktikum: Simulation einer globalen Firma als verteiltes System

Zusammenfassung Verteilte Systeme

- In der Praxis gibt es nur verteilte Systeme
- „Verteiltheit“ ist eine Herausforderung.
  Vergessen Sie niemals eine der Pitfalls!
- Auch wenn die Wahrscheinlichkeit eines Fehlers extrem selten ist (z.B. 1:1 Milliarde), tritt der Fehler in der Praxis bei großen Systemen dann sehr oft ein!



QUIZ

https://arsnova.thm.de/p/06477742

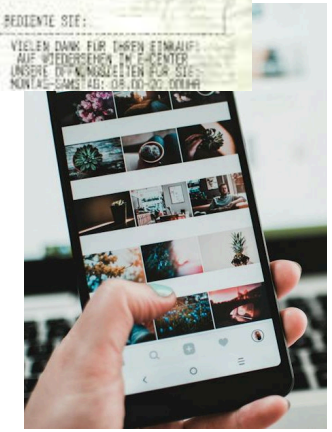


Agenda

1.	Einleitung	2
2.	Versionsverwaltung	17
3.	Betriebssysteme	48
a.	Dateisystem	69
b.	Prozesse	77
c.	Speicherverwaltung	86
d.	Virtualisierung und Containerisierung	95
4.	Verteilte Systeme	102
5.	REST-Services	134
6.	**Datenbanken**	**203**
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343

Daten sind überall



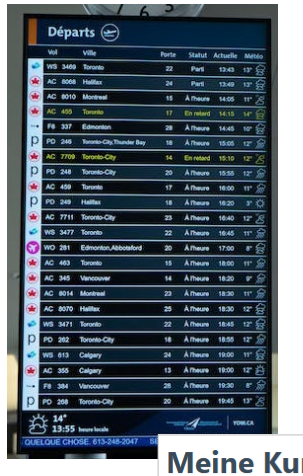






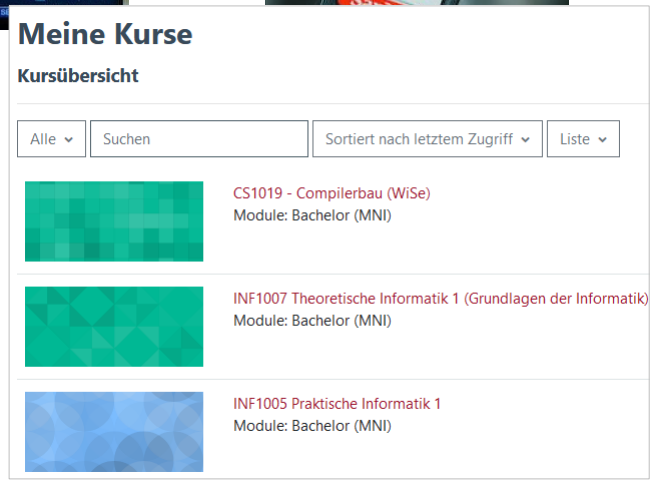










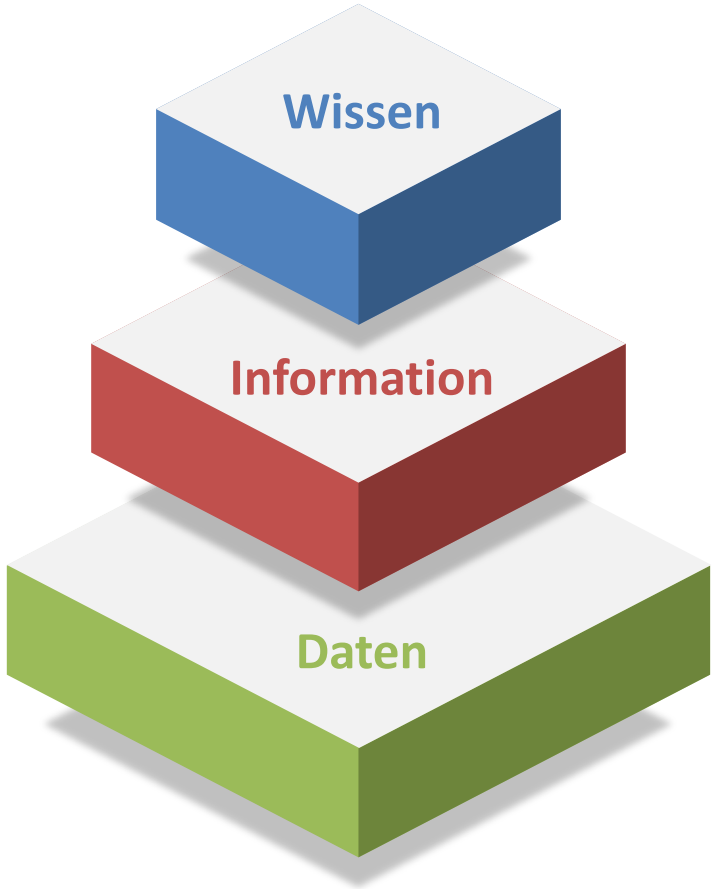


Daten -> Information -> Wissen

- **Daten** sind das „Rohmaterial“ und repräsentieren Informationen
- Daten sind strukturiert und basieren auf einem festen Zeichenvorrat

- **Informationen** dienen dem Verständnis der Daten
- Informationen sind zweckorientiert

- **Wissen** entsteht durch systematische Verknüpfung von Informationen
- Wissen ist subjektiv und personenbezogen



Fakten

- Fakten sind Informationen, die als objektiv wahr gelten und überprüfbar sind

- Beispiel
  - Berlin, 20°, 3.12.2025, 12:00 CEST — Datum
  - Heute um 12 Uhr beträgt die Temperatur in Berlin 20° — Fakt
  - Es ist angenehm warm — Information
  - Bei 20° kann ich den Pullover zuhause lassen — Wissen

Behandlung von Daten

Welche Daten besitzen Sie?
Was machen Sie mit den Daten?

- Speichern
- Abrufen
- Löschen
- Erzeugen
- Ändern
- Aufbewahren
- …

